GSoC 2026 Proposal: Multimodal AI and Agent API Eval Framework

About

1. **Full Name:** Jiayu Zhao
2. **Contact info:** jiayuzhao@uchicago.edu
3. **GitHub profile link:** https://github.com/jiayuzhao05
4. **Twitter, LinkedIn, other socials:** https://www.linkedin.com/in/fenney-zhao-74642a1b9/
5. **Time zone:** UTC -5

University Info

1. **University name:** university of chicago
2. **Program:** ms in computational analytics and public policy
3. **Year:** 2nd
4. **Expected graduation date:** 2026

Problem

Developers testing AI APIs have no unified way to systematically evaluate and compare model outputs across text, image, and voice modalities. Benchmarking tools like lm-evaluation-harness and lighteval are powerful but CLI-only, and comparing results across providers requires manual work. API Dash already excels at sending AI API requests and previewing responses, and the missing piece is a structured evaluation layer.

Solution

I will build an end-to-end Multimodal AI and Agent API Eval Framework for API Dash, consisting of a FastAPI evaluation and a React dashboard. The backend provides a provider abstraction layer supporting multiple AI services, a pluggable metrics engine , dataset management), and integration with standard benchmark runners. Server-Sent Events stream real-time evaluation progress to the frontend. The dashboard offers an intuitive interface for configuring evaluation runs, monitoring execution live, and visualizing comparative results through leaderboards, charts, and per-sample inspection. The entire system is dependency-lite, no Redis or Celery, just pip install and npm install.

Deliverables: FastAPI evaluation engine with provider abstraction and SSE streaming Dataset loader supporting local files, HuggingFace datasets, and remote URLs across text, image, and audio

formats Pluggable metrics engine covering text, image, voice, and agent evaluation benchmarks llm-evaluation-harness and lighteval benchmark runner integration React/TypeScript dashboard with configuration panel, real-time execution monitor, and interactive results visualization

Project Proposal Information

1. Proposal Title

Multimodal AI and Agent API Eval Framework

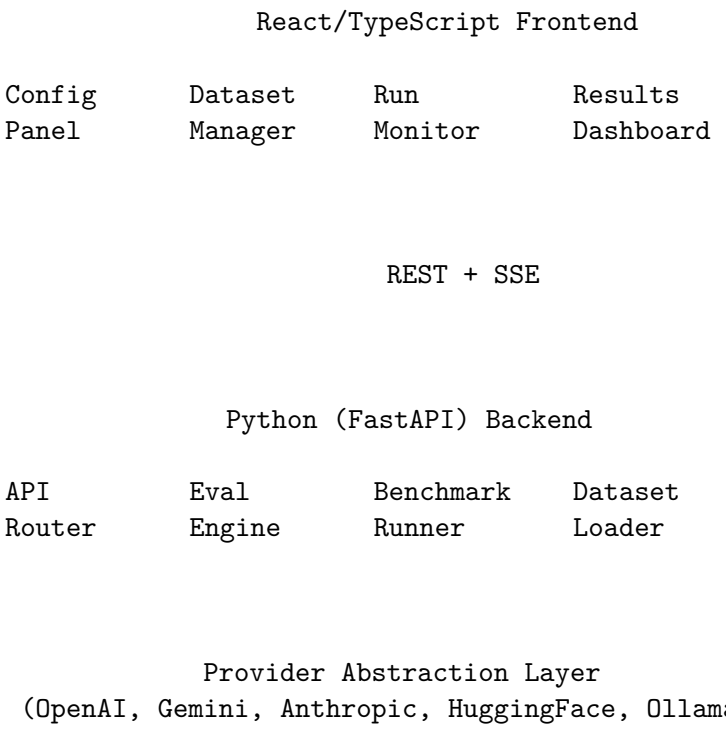
2. Abstract

AI APIs are proliferating rapidly, but developers lack a unified, intuitive tool to systematically evaluate and compare AI model outputs across text, image, and voice modalities. This project proposes building an end-to-end Multimodal AI and Agent API Eval Framework for API Dash — a React/TypeScript frontend dashboard backed by a Python (FastAPI) evaluation engine. The framework will enable developers to configure AI API requests, input test/custom datasets, run standardized benchmarks (lm-evaluation-harness, lighteval), and view comparative evaluation results through interactive visualizations. The system uses Server-Sent Events (SSE) for real-time progress streaming and supports both local file uploads and remote dataset URLs.

3. Description

3.1 Architecture Overview

The framework follows a three-layer architecture designed for modularity, extensibility, and developer experience:



```
SQLite (Job State & History)
```

3.2 Python FastAPI Evaluation Engine

Why FastAPI: Lightweight, async-first, native SSE support, perfect for streaming long-running benchmarks. Zero heavy infrastructure dependencies (no Redis/Celery) — just `pip install`.

Provider Abstraction Layer A unified interface for interacting with multiple AI API providers:

```python
class AIProvider(ABC):
    @abstractmethod
    async def complete(self, request: EvalRequest) -> EvalResponse:
        """Send a request to the AI provider and return structured response."""
        pass

    @abstractmethod
    def supports_modality(self, modality: Modality) -> bool:
        """Check if provider supports text/image/voice modality."""
        pass

class OpenAIProvider(AIProvider):
    async def complete(self, request: EvalRequest) -> EvalResponse:
        # Handle text, vision, and audio API calls
        ...

class GeminiProvider(AIProvider):
    async def complete(self, request: EvalRequest) -> EvalResponse:
        # Handle Gemini multimodal requests
        ...
```

Supported providers: OpenAI, Google Gemini, Anthropic Claude, HuggingFace Inference, Ollama (local).

Evaluation Engine The core engine orchestrates evaluation runs:

```python
class EvalEngine:
    async def run_evaluation(
        self,
        config: EvalConfig,
        dataset: Dataset,
        providers: List[AIProvider],
    ) -> AsyncGenerator[EvalProgress, None]:
        """Execute evaluation and yield progress updates via SSE."""
        for i, sample in enumerate(dataset.samples):
            for provider in providers:
```

```
                result = await provider.complete(sample.to_request())
                score = self.metric_engine.score(sample.expected, result)
                yield EvalProgress(
                    current=i, total=len(dataset.samples),
                    provider=provider.name, score=score
                )
```

Benchmark Integration Wrap existing tools as subprocess runners with structured output parsing:

```
class LMHarnessRunner:
    async def run(self, config: BenchmarkConfig) -> AsyncGenerator[str, None]:
        process = await asyncio.create_subprocess_exec(
            "lm_eval", "--model", config.model,
            "--tasks", config.task, "--output_path", config.output_dir,
            stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.PIPE
        )
        async for line in process.stdout:
            yield line.decode()  # Stream to frontend via SSE

class LightEvalRunner:
    async def run(self, config: BenchmarkConfig) -> AsyncGenerator[str, None]:
        # Similar subprocess wrapper for lighteval
        ...
```

Dataset Management Support multiple input sources:

- **Local file upload**: CSV, JSON, JSONL files with configurable column mapping
- **Remote URLs**: HuggingFace datasets, S3/GCS links
- **Custom inline**: Manual input through the UI
- **Multimodal**: Text files, image files (base64 or URL), audio files

```
class DatasetLoader:
    async def load(self, source: DatasetSource) -> Dataset:
        if source.type == "local":
            return self._load_local(source.path, source.format)
        elif source.type == "huggingface":
            return self._load_hf(source.repo_id, source.split)
        elif source.type == "url":
            return self._load_url(source.url)
```

Metrics Engine Pluggable metric system supporting standard and custom metrics:

Modality	Metrics
Text	Exact Match, BLEU, ROUGE, BERTScore, Perplexity
Image (Captioning)	BLEU, CIDEr, CLIPScore
Image (Classification)	Accuracy, F1, Confusion Matrix

Modality	Metrics
Voice (STT)	Word Error Rate (WER), Character Error Rate (CER)
Voice (TTS)	MOS (Mean Opinion Score), Naturalness
Agent	Tool-call Accuracy, Step Completion Rate, Trajectory Match

Custom metrics via a plugin interface:

```python
class CustomMetric(ABC):
    @abstractmethod
    def score(self, expected: Any, actual: Any) -> float:
        pass
```

```python
@app.get("/api/eval/{run_id}/stream")
async def stream_eval_progress(run_id: str):
    async def event_generator():
        async for progress in eval_engine.get_progress(run_id):
            yield {
                "event": "progress",
                "data": json.dumps(progress.dict())
            }
        yield {"event": "complete", "data": json.dumps(final_results)}
    return EventSourceResponse(event_generator())
```

SSE Streaming for Real-time Updates

3.3 React Dashboard

Built with React + TypeScript + Vite for fast development and lightweight bundle.

UI Components **1. Configuration Panel** - Model/provider selection (multi-select for comparison) - Parameter configuration (temperature, max_tokens, top_p, etc.) - API key management (encrypted local storage) - Benchmark selection (lm-harness tasks, lighteval tasks, custom)

2. Dataset Manager - Drag-and-drop file upload - HuggingFace dataset browser - Custom dataset builder with schema validation - Column mapping for CSV/JSON files - Preview first N samples before running

3. Execution Monitor (Real-time) - Live progress bar per provider - Streaming log output (SSE-driven) - Per-sample result table (populating in real-time) - Estimated time remaining - Cancel/pause support

```typescript
interface EvalConfig {
  providers: ProviderConfig[];
  benchmark?: string;
  dataset: DatasetConfig;
  metrics: MetricConfig[];
```

```
  parameters: Record<string, any>;
}

interface EvalProgress {
  runId: string;
  current: number;
  total: number;
  provider: string;
  latestScore?: number;
  elapsedMs: number;
  estimatedRemainingMs?: number;
}

interface EvalResult {
  runId: string;
  provider: string;
  metrics: Record<string, number>;
  samples: SampleResult[];
  metadata: RunMetadata;
}

interface SampleResult {
  input: MultimodalInput;
  expected?: string;
  actual: string;
  scores: Record<string, number>;
  latencyMs: number;
}
```

Key TypeScript Interfaces

3.4 Multimodal Support

Text Evaluation Standard NLP benchmarks: MMLU, HellaSwag, ARC, TruthfulQA, GSM8K. Custom prompt datasets with expected output comparison.

Image Evaluation

- **Captioning**: Input image → model generates caption → compare with reference using BLEU/CIDEr
- **VQA**: Input image + question → model answers → compare with ground truth
- **Classification**: Input image → model classifies → accuracy/F1
- **Generation**: Text prompt → model generates image → CLIPScore comparison

Voice Evaluation

- **Speech-to-Text (STT)**: Input audio → model transcribes → WER/CER against reference
- **Text-to-Speech (TTS)**: Input text → model generates audio → automated quality metrics

Agent Evaluation

- **Tool-call Accuracy**: Does the agent call the correct tools with correct parameters?
- **Multi-step Workflow**: Track full conversation traces, evaluate intermediate steps
- **Task Completion**: End-to-end success rate on predefined tasks

3.5 Security & Privacy

- API keys stored with encryption in local browser storage (never sent to our backend except for forwarding to providers)
- Sensitive headers sanitized in logs
- All evaluation runs are local — no telemetry or data collection
- Support for local models (Ollama) for fully offline evaluation

3.6 Integration with API Dash

The eval framework can integrate with API Dash's existing AI request feature: - Import API configurations from API Dash collections - Share evaluation results back to API Dash workspace - Use API Dash's environment variables for provider API keys

4. Weekly Timeline

Community Bonding Period (Weeks 0-1)

- Deep-dive into API Dash codebase, especially `genai` package and AI request handling
- Set up development environment with Python backend + React frontend
- Finalize API contracts (TypeScript interfaces + Python Pydantic models) with mentors
- Create project board and milestones on GitHub

Week 1-2: Backend Foundation

- Implement FastAPI project scaffolding with proper project structure
- Build Provider Abstraction Layer (OpenAI, Gemini, Anthropic)
- Implement Dataset Loader (local file upload + JSON/CSV parsing)
- Add SQLite for job state management
- **Deliverable**: Backend can receive eval config, load dataset, send requests to AI providers

Week 3-4: Evaluation Engine & Metrics

- Implement core Evaluation Engine with async execution
- Build Metrics Engine with text metrics (Exact Match, BLEU, ROUGE)
- Add SSE streaming for real-time progress updates
- Implement lm-evaluation-harness subprocess runner
- **Deliverable**: End-to-end text evaluation pipeline with streaming progress

Week 5-6: Frontend

- Build React/TypeScript project with Vite
- Implement Configuration Panel (provider selector, parameters, API keys)
- Build Dataset Manager (file upload, data preview, column mapping)
- Create Execution Monitor with SSE integration (live progress, log stream)

- **Deliverable**: Full frontend config → execution flow working with backend

Midterm Evaluation

Week 7-8: Results Dashboard & Image Evaluation

- Build Results Dashboard (leaderboard, charts, sample inspector)
- Add image evaluation support (captioning, VQA, classification)
- Implement CLIPScore and image-specific metrics
- Add HuggingFace dataset browser integration
- **Deliverable**: Complete results visualization + image evaluation support

Week 9-10: Voice Evaluation & Agent Evaluation

- Add voice/audio evaluation support (STT with WER/CER)
- Implement agent evaluation (tool-call tracking, multi-step workflows)
- Add lighteval benchmark runner integration
- Build export functionality (CSV, JSON, PDF reports)
- **Deliverable**: Full multimodal evaluation support (text + image + voice + agent)

Week 11: Integration & Polish

- Integrate with API Dash (import configs, share results)
- Add evaluation history with comparison features
- Implement custom metric plugin system
- Performance optimization and caching
- **Deliverable**: Polished, integrated evaluation framework

Week 12: Testing, Documentation & Submission

- Write comprehensive unit and integration tests
- Create user documentation with examples
- Write developer documentation for extending the framework
- Record demo video
- Final code review and PR submission
- **Deliverable**: Production-ready eval framework with full documentation